SEG 2105 - Fall 2026

Assignment 2

Requirements and Object-Oriented Modelling

O-Connect City Transit Management System

Student Name	Student Number
Jahnavi Kasu	300489250
Fnu Gurneet Kaur	300515777

Submitted: November 6th, 2026

Contents

Question 1 – UML class diagram (domain model)

Question 2 – UML object diagram

Question 3 – Functional requirements

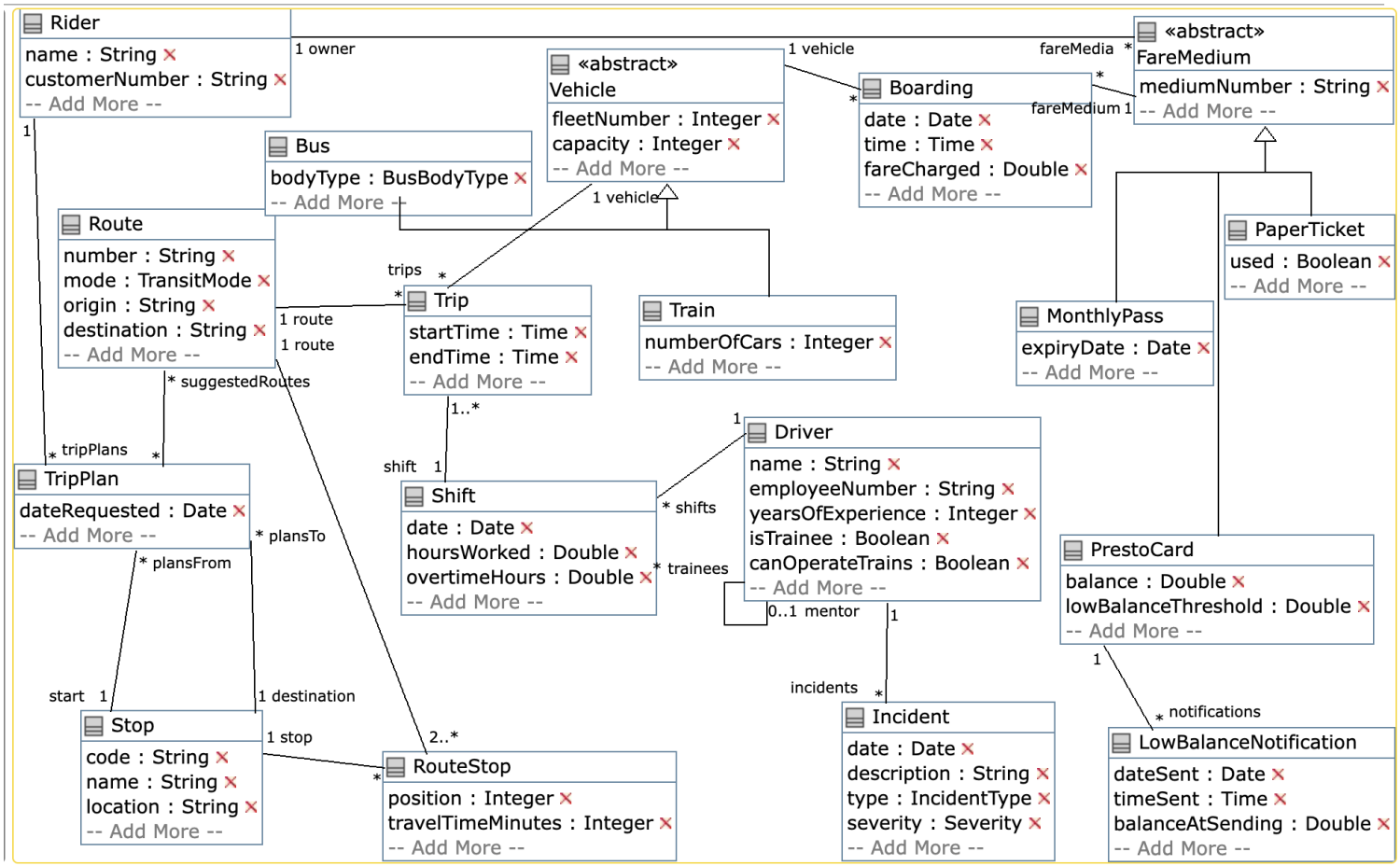
Question 4 – Non-functional requirements

Question 5 – UML use case diagram

Question 6 – Detailed use case description

Bonus – Umple model (source file OConnect.ump submitted separately)

Question 1: UML class diagram (domain model)



Constraint on the mentor association: {a driver with isTrainee = true has exactly 1 mentor; a driver with isTrainee = false has 0 mentors; a mentor always has isTrainee = false}. The multiplicity is 0..1 because the association is between Driver and Driver, so it must allow senior drivers to have no mentor; the constraint enforces “exactly one” for trainees.

Question 1: Notes and assumptions

The class diagram was generated by Umple from the model file OConnect.ump (submitted for the bonus). Umple shows the association classes RouteStop and Trip as classes linked to both ends of their association. Abstract classes are marked «abstract». Getters, setters and constructors are omitted.

The attributes isTrainee, canOperateTrains, overtimeHours, origin and destination are derived (calculated) attributes. In standard UML notation they would be written /isTrainee : Boolean, /canOperateTrains : Boolean, /overtimeHours : Double, /origin : String and /destination : String. Umple's diagram does not display the "/", but in OConnect.ump they are declared as derived attributes with their formulas. Umple shows enumeration types as attribute types; their values are: TransitMode {Bus, OTrain}, BusBodyType {Standard, Articulated, DoubleDecker}, IncidentType {Accident, Fine}, Severity {Minor, Major}.

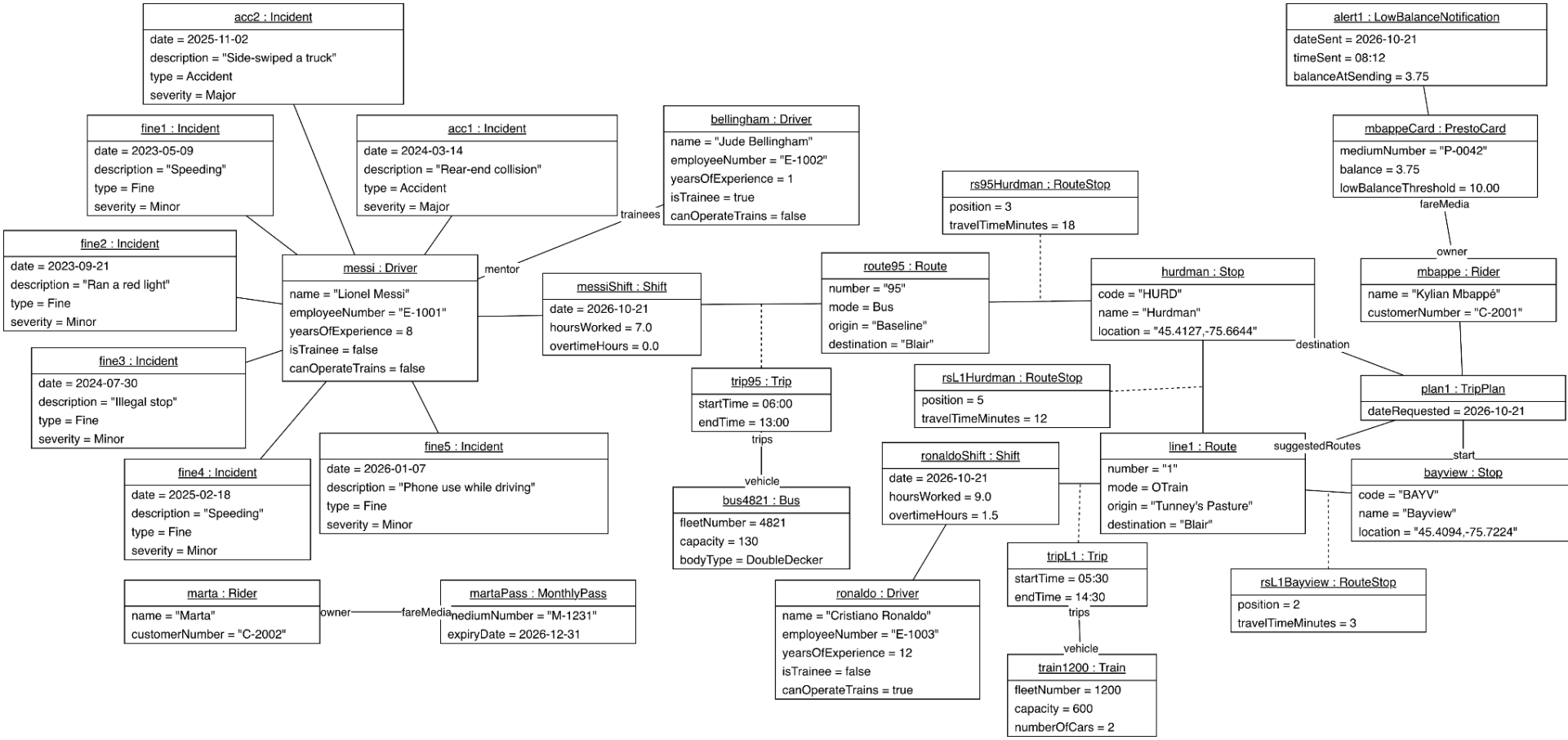
1. **Routes and stops.** A route serves an ordered list of at least two stops and a stop can be served by many routes. Because the position and the scheduled travel time belong to a particular (route, stop) pair, they are modelled in the association class **RouteStop**. The route's point of origin and destination are not stored separately (that would be redundant); they are the derived attributes **origin** (stop at position 1) and **destination** (stop at the last position).
2. **Bus route vs. O-Train line.** Both kinds of route have exactly the same attributes and associations, so they are distinguished by the enumeration attribute **mode : TransitMode** rather than by subclasses (a generalization would add no new attributes or behaviour). Constraint: a Trip on a route whose mode is Bus must use a Bus; a Trip on an OTrain line must use a Train.

3. **Vehicles.** Bus and Train **are** subclasses of the abstract class Vehicle because each has its own attribute (bodyType, numberOfCars). Body type is an enumeration (Standard, Articulated, DoubleDecker).
4. **Drivers.** Trainee, senior and train-authorized are not subclasses, because a driver moves between these categories over time as experience grows. They are the derived attributes **isTrainee** (yearsOfExperience < 2) and **canOperateTrains** (yearsOfExperience ≥ 10). Constraints: a trainee has exactly one mentor and a non-trainee has none (hence 0..1 on the mentor end); a mentor is not a trainee; a driver can only be assigned to a Trip on an OTrain line if canOperateTrains = true.
5. Driver is the only kind of employee described, so no Employee superclass is introduced (a superclass with a single subclass would be an unnecessary generalization). Riders and drivers only share a name, so there is no Person superclass either.
6. **Shifts and overtime.** A **Shift** is one working day of one driver; the system records **hoursWorked** for that day, and **overtimeHours** = max(0, hoursWorked − 7.5).
7. **Trips (route assignments).** Each time a driver is assigned to a route during a shift (the first assignment of the day or a later re-assignment) a **Trip** is created: it is the association class between Shift and Route and records the start and end time and the vehicle used. A shift has at least one trip. We assume a driver covers the same route at most once per shift.
8. **Incidents.** Accidents and fines have identical attributes, so the kind of incident is the enumeration **type : IncidentType** (Accident, Fine) instead of two subclasses; severity is the enumeration Severity (Minor, Major). Each incident record belongs to exactly one driver.
9. **Fare media.** PaperTicket, MonthlyPass and PrestoCard are subclasses of the abstract class FareMedium because each has

different attributes and behaviour. Each fare medium belongs to one rider (tickets are assumed to be linked to the buyer's account); a rider may own several. reload() and deductFare() update the Presto card balance; they are defined in OConnect.ump but Umple does not display operations in the diagram.

10. **Low-balance notifications.** When a deduction makes the balance fall below lowBalanceThreshold, a LowBalanceNotification is created for that card. A threshold of 0.00 means the rider has not set one.
11. **Boarding.** Each use of a fare medium to board a vehicle is recorded as a Boarding (date, time, fare charged). It is a normal class, not an association class, because the same card can board the same vehicle many times. The fare table itself is outside the scope of this model.
12. **Trip planning.** A TripPlan records a rider's request: a start stop, a destination stop and the route(s) the system suggests. Start and destination are assumed to be stops, not street addresses.
13. Money is stored as Double (dollars), location as a String of GPS coordinates; Date and Time are the Umple/UML primitive types.

Question 2: UML object diagram



Question 2: Assumptions

1. The scenario takes place on 2026-10-21; both shifts are on that day.

2. Lionel Messi has 8 years of experience: at least 2, so he may mentor Jude Bellingham, and fewer than 10, which is consistent with him driving a bus.
3. Messi's 7-hour shift is a single Trip on route 95 (06:00–13:00) with bus #4821; Ronaldo's 9-hour shift is a single Trip on Line 1 (05:30–14:30) with train #1200. Neither was re-assigned that day. Ronaldo's overtimeHours = 9.0 − 7.5 = 1.5.
4. Messi's 7 incidents are shown as 7 Incident objects: 2 accidents with severity Major and 5 fines, which we assume are Minor. Ronaldo has no Incident objects (clean record).
5. Employee numbers, customer numbers, card/pass numbers, stop codes, coordinates, vehicle capacities, times, and incident dates/descriptions are invented, since the scenario does not give them.
6. Only the RouteStop links relevant to the scenario are drawn: Hurdman on route 95 (3rd stop, 18 min) and on Line 1 (5th stop, 12 min), and Bayview on Line 1 (2nd stop, 3 min, assumed). The other stops of each route exist but are omitted for readability; the end points of both routes appear through the derived attributes origin and destination.
7. For Mbappé's trip plan from Bayview to Hurdman, the system suggests O-Train Line 1, since it serves both stops in that order.
8. Mbappé's low-balance notification was sent when his balance became $3.75, below his $10.00 threshold. Marta's monthly pass expires on 2026-12-31.
9. Both association classes in the class diagram are instantiated: RouteStop (3 instances) and Trip (2 instances).

Question 3: Functional Requirements

FR1 - Trip Planning: The system shall provide a rider with the route or combination of routes connecting their specified starting point to their chosen destination when they request a trip plan.

FR2 - Driver Route Assignment: The system shall allow a dispatcher to reassign a driver to a different route during the driver's shift.

FR3 - Work-Hour Tracking: The system shall record the number of hours worked by each driver on a given day.

FR4 - Incident Management: The system shall record every incident involving a driver, including the incident date, description, type (accident or fine), and severity (major or minor).

FR5 - Low-Balance Notifications: The system shall automatically notify a rider when their Presto card balance falls below the low-balance threshold configured on their card.

Question 3: Assumptions

1. For trip planning, the rider's starting point and destination are stops in the O-Connect network.

2. Driver assignments and reassignments are associated with the workday on which they occur.

3. Each recorded driver incident is classified as either an accident or a fine and is assigned a severity of either major or minor.

4. The system evaluates a Presto card's balance against its configured low-balance threshold to determine when a notification must be sent.

Question 4: Non-Functional Requirements

Quality Requirements:

NFR1 - Performance: The O-Connect system shall display trip-planning results within 2 seconds for at least 95% of requests under normal operating conditions with up to 500 concurrent active users.

NFR2 - Availability: The O-Connect system shall maintain an availability of at least 99.5% per calendar month, excluding scheduled maintenance periods.

NFR3 - Security: The system shall only allow users with the Dispatcher or Operations Manager role to view driver incident records, and shall deny all other users access.

NFR4 - Usability: At least 90% of first-time riders participating in usability testing shall be able to successfully plan a trip between two stops within 3 minutes without assistance.

NFR5 - Recoverability: Following a system failure, the O-Connect system shall be capable of restoring its stored data and resuming normal operations within 2 hours, with no more than 24 hours of data loss.

Platform Requirements:

NFR6 - Web Browser Compatibility: The O-Connect system shall support the two most recent major versions of Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari.

NFR7 - Operating System Compatibility: The O-Connect system shall be accessible through supported web browsers on devices running Windows 11 or later, macOS 14 or later, Android 13 or later, and iOS 17 or later.

Process Requirements:

NFR8 - Code Review: Every source code change shall be reviewed and approved by at least one other development team member before being merged into the main codebase.

NFR9 - Software Testing: Before each software release, the development team shall execute automated unit and integration tests, and all tests designated as critical shall pass before deployment.

Question 4: Assumptions - The following assumptions establish the technical and operational conditions used to define the non-functional requirements:

1. System Architecture: The O-Connect system will be developed as a web-based application rather than requiring users to install a dedicated desktop application. This assumption supports the browser and operating system compatibility requirements.

2. Expected System Usage: O-Connect is expected to serve thousands of daily riders and transit employees. A target of 500 concurrent active users is assumed for normal peak operating conditions when evaluating system performance. This is a proposed capacity target, not a figure specified in the assignment.

3. Performance Expectations: Trip-planning results should be delivered within 2 seconds for at least 95% of requests under the assumed normal peak load. The performance target is intended to provide riders with timely access to route information.

4. Availability Expectations: Since O-Connect supports public transit operations, the system is expected to remain accessible through its operational periods. Scheduled maintenance is excluded from the monthly availability calculation, provided it is planned in advance and communicated to affected users.

5. Information Confidentiality: Rider information, driver information, employee records and incident records are considered confidential. Users must authenticate themselves and have appropriate permissions before accessing restricted information.

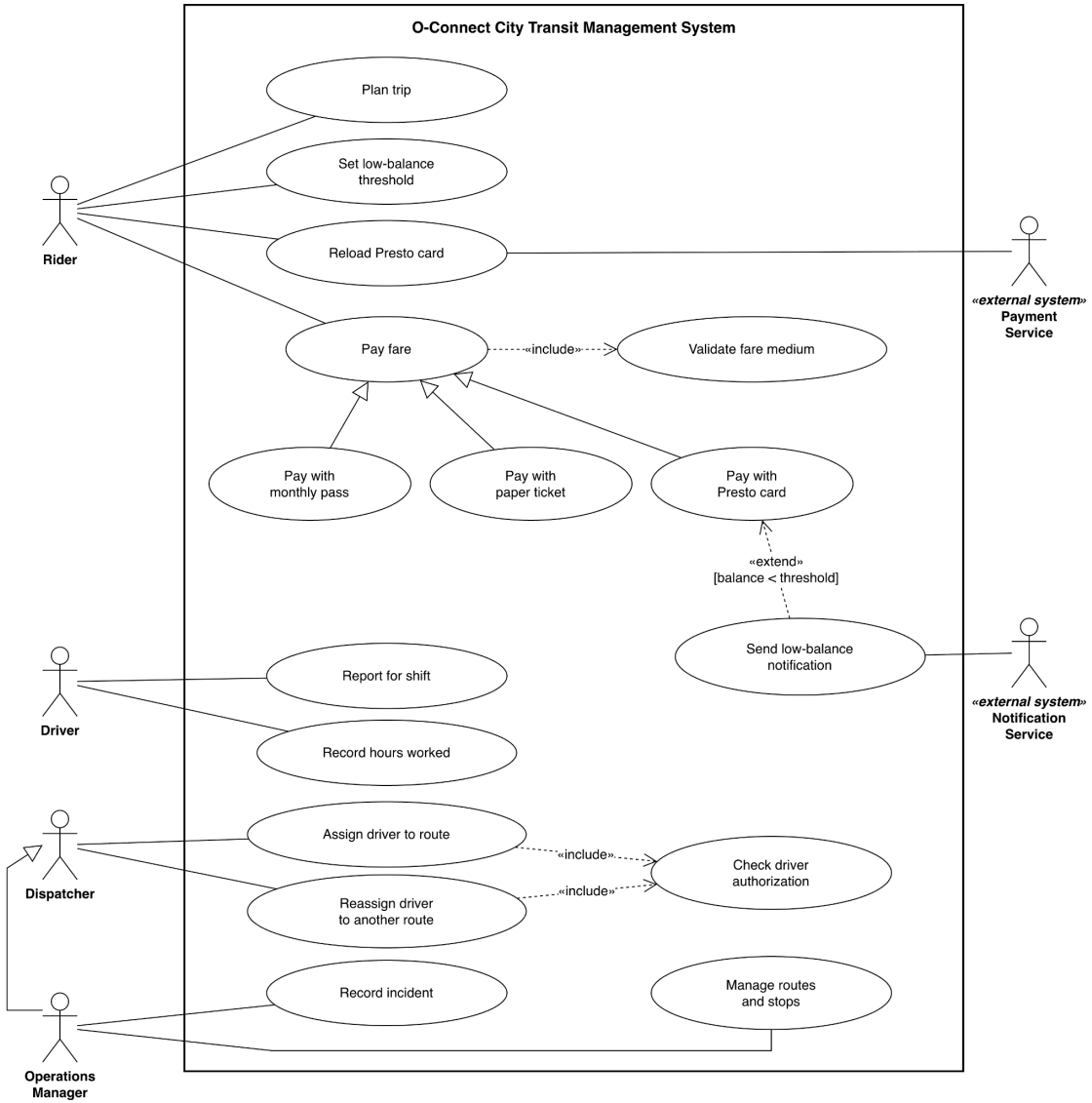
6. Usability Evaluation: Usability will be evaluated through testing with representative first-time riders. Participants will be given basic instructions before attempting to plan a trip, and their completion time and need for assistance will be recorded.

7. Data Recovery: The system will maintain regular backups of its stored information. A maximum recovery time of 2 hours and a maximum acceptable data loss of 24 hours are assumed as recovery targets following a system failure.

8. Platform Support: The minimum operating system versions listed in NFR 7 are proposed compatibility targets. The system will be tested using the supported operating system and browser combinations.

9. Development Practices: The development team will use a shared version-control repository with a main codebase. The source code changes will be reviewed before merging, and automated unit and integration tests will be maintained and executed before each release.

Question 5: UML use case diagram



Question 5: Assumptions

1. Primary actors (left): Rider, Driver, Dispatcher and Operations Manager. The Operations Manager is a specialization of the Dispatcher (actor generalization): it can do everything a dispatcher does, and also manage routes and stops and record incidents.
2. Secondary / external actors (right): the Payment Service (external bank/card processor that handles the payment when a Presto card is reloaded) and the Notification Service (external SMS / e-mail / push gateway).
3. Use case generalization: Pay with Presto card, Pay with monthly pass and Pay with paper ticket are specializations of Pay fare.
4. «include»: every fare payment always validates the fare medium; assigning and re-assigning a driver always checks the driver's authorization (trains require 10+ years of experience).
5. «extend»: Send low-balance notification extends Pay with Presto card only when the new balance is below the rider's threshold.
6. The card readers on vehicles are part of the O-Connect system, so they are not shown as actors. The driver records the hours worked at the end of the shift.

Question 6: Detailed Use Case Description

A. Name

Pay with Presto card

B. Actors

- Primary Actor: Rider

- Secondary Actor: Notification Service (external system)

C. Goals

The rider wants to pay the applicable transit fare using their Presto card when boarding a bus or train.

O-Connect wants to collect the correct fare, maintain an accurate record of every boarding, and notify the riders when their Presto card balance falls below their configured threshold.

D. Preconditions

1. The rider has a Presto card associated with their rider account in the O-Connect System.

2. The O-Connect system and the vehicle's card reader are operational.

Trigger: The rider taps their Presto card on the vehicle's card reader to pay the transit fare.

E. Summary

This use case describes how a rider pays their transit fare using a Presto card. The system validates the card, checks whether it has sufficient balance, deducts the applicable fare and records the boarding. If the remaining balance falls below the rider's configured

low-balance threshold, the system sends a notification through the external Notification Service.

F. Related Use Cases

1. Pay fare: Generalized use case of which Pay with Presto card is a specialization.

2. Validate fare medium: Included by Pay fare and inherited by this specialized use case.

3. Send low-balance notification: Extends Pay with Presto card when the remaining balance falls below the configured threshold.

G. Steps

Main Success Scenario (Basic Flow)

Actor (Rider)	System (O-Connect)
1. The rider taps their Presto card on the vehicle's card reader.	2. The system reads the Presto card information.
	3. The system validates the Presto card.
	4. The system determines the applicable fare.
	5. The system checks whether the card has sufficient balance to cover the fare.
	6. The system deducts the applicable fare from the card balance.

	7. The system records the boarding, including the date, time and fare charged.
	8. The system confirms that the payment was successful.
9. The rider boards the vehicle.	

Alternative Flows

Invalid Presto card - Step 3

Actor (Rider)	System (O-Connect)
	1. The system determines that the Presto card is invalid or cannot be validated.
	2. The system rejects the payment and informs the rider that the card is invalid.
3. The rider ends the payment attempt.	

The use case ends without deducting any fare or creating a successful boarding record.

Insufficient card Balance - Step 5

Actor (Rider)	System (O-Connect)
	1. The system determines that the card balance is lower

	than the applicable fare.
	2. The system rejects the payment and informs the rider that the card has insufficient funds
3. The rider ends the payment attempt.	

The use case ends without deducting any fare or creating a successful boarding record.

Card Cannot Be Read - Alternative to Step 2

Actor (Rider)	System (O-Connect)
	1. The system determines that it cannot read the Presto card information.
	2. The system informs the rider that the card could not be read and requests another tap.
3. The rider taps the Presto card again.	
	4. The system resumes the main success scenario at Step 2

Low-Balance Notifications (After Step 6)

Actor (Rider)	System/External Service
	1. The system determines that the remaining card balance is

	below the rider's configured low-balance threshold.
	2. The system generates a low-balance notification containing the current card balance.
	3. The system sends the notification through the external Notification Service.
	4. The Notification Service delivers the notification to the rider.

The use case continues with Step 7 of the main success scenario.

Notification Service Unavailable - Alternative to Step 3 of Low-Balance Notifications

1. The system determines that the Notification Service is unavailable or cannot deliver the notification.
2. The system records the notification delivery failure.

The fare deduction and boarding remain successful. The use case continues with Step 7 of the main success scenario.

H. Postconditions

Successful Completion:

1. The applicable fare has been deducted from the rider's Presto card balance.
2. A boarding record has been created containing the date, time and fare charged.
3. If the remaining balance is below the configured low-balance threshold, a low-balance notification containing the

current card balance is generated and sent to the external Notification Service for delivery to the rider. If the delivery fails, the notification delivery failure is recorded.

Unsuccessful Completion:

1. No fare has been deducted from the rider's Presto card.
2. No successful boarding record has been created.

Question 6: Assumptions

1. The rider's Presto card is associated with their rider account in the O-Connect system, allowing the system to identify the card owner.

2. The applicable transit fare is determined using a fare structure maintained by O-Connect. The assignment does not specify the fare amounts or calculation rules.

3. A Presto card must have a balance equal to or greater than the applicable fare for payment to succeed.

4. A card balance exactly equal to the applicable fare is considered sufficient.

5. The low-balance threshold is checked after the fare has been deducted, using the remaining card balance.

6. A notification is triggered only when the remaining balance is strictly below the configured threshold. A threshold of $0.00 indicates that the rider has not configured a low-balance threshold.

7. The external Notification Service is responsible for delivering low-balance notifications to riders.

8. Failure to deliver a low-balance notification does not reverse a successful fare deduction or boarding transaction.

9. The vehicle’s card reader is considered part of the O-Connect system, consistent with the assumption in Question 5.